

Artificial Intelligence-assisted Spacecraft Health Monitoring and Decision Support System - ASCLEPIUS

Introduction

With the ever-increasing capabilities of artificial intelligence algorithms and their rapid worldwide adoption across modern engineering systems, the domain of spaceflight stands out as one in which AI could bring about a revolutionary acceleration of humanity's knowledge of our universe as well as our physical reach beyond Earth. By enhancing autonomy, fault-detection, and operational decision-making, AI-driven technologies have the potential to improve spacecraft resilience and expand the capabilities of space missions. While space missions have historically employed deterministic rule-based thresholding methods to detect anomalous telemetry data, this technique has certain limitations such as the time cost required to manually set and tune the thresholds for countless individual variables, and as such, AI-assisted methods are becoming essential for maximising operational effectiveness in complex missions.

In this project we develop Asclepius - a software system for the purpose of spacecraft health monitoring, combining rule-based and machine learning anomaly-detection methods to identify, classify, and record health-threatening events in generated spacecraft telemetry, integrated with an LLM-based decision support engine to provide mission level assessments, prioritise detected events, and assist the spacecraft operator in responsive actions. We validate the Asclepius system's utility using synthetic telemetry generated to simulate three space mission types: Earth Observation, Lunar Exploration, and Deep Space. The overall architecture of the system is illustrated in Figure 1.

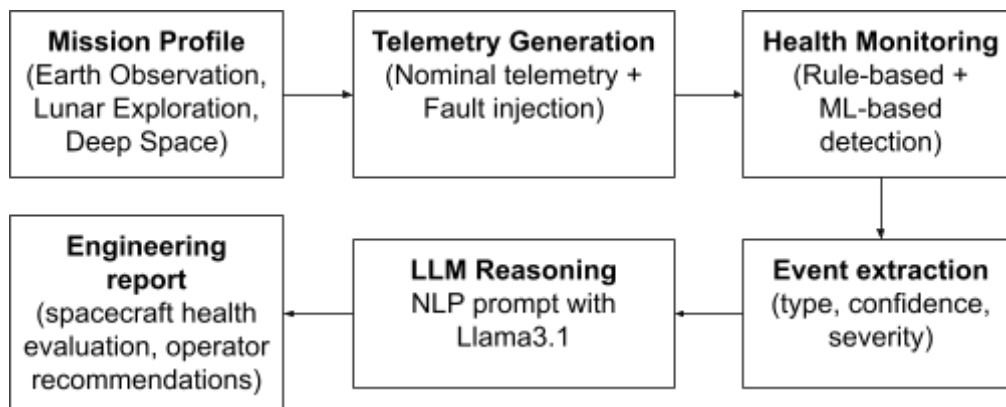


Figure 1: system architecture flowchart

Telemetry Generation

In order to create a more controlled and customisable environment for the application of the spacecraft system, we chose to generate synthetic data representing the telemetry variables characteristic of modern space missions. This process involved first generating the nominal telemetry based on likely normal operating ranges for each variable, followed by the injection of

a number of faults characterised by anomalous telemetry readings, whose presence we tested if the system could followingly detect and identify.

The nominal telemetry framework consisted of defining normal operating ranges for several variables within six core subsystems: Power, Thermal Control, Attitude and Orbital Control (AOCS), Communication, Propulsion, and On-board Computer (OBC). The telemetry variables included were as follows: Power - battery voltage, battery current, battery state of charge, solar array power; Thermal Control - battery temperature, bus temperature, payload temperature; AOCS - reaction wheel speed, gyroscope drift; Coms - communication signal, data rate, packet loss; Propulsion - fuel level, thruster temperature, thrust level; OBC - computer load, memory usage.

In addition to this core telemetry, further variables were incorporated into the data specific to each of the three missions based on an evaluation of the unique features of each mission type. For the Earth Observation scenario, downlink rate and ground station visibility were added to the Communication subsystem, while pointing error and attitude accuracy were added to the AOCS subsystem. An additional Optics subsystem was also created for this mission, given that this is an integral part of earth observation payloads, and contained data for camera temperature, image quality and imaging power. For the Lunar Exploration scenario, a battery state of charge variable was added to the Power subsystem, radiator temperature and heater power to the Thermal subsystem, while earth visibility and link margin variables complemented the core Communication telemetry. Furthermore, a separate Mobility subsystem was created specifically for this mission to describe the vehicle's movement on the lunar surface, and included data on wheel slip ratio, terrain slope, and surface speed. Lastly, for the Deep Space mission scenario, telemetry for the output of a radioisotope thermoelectric (RTG) generator (a nuclear-powered battery often used in deep-space missions operating in regions where sunlight is too faint to generate power) was added to the core Power subsystem, antenna status and signal-to-noise ratio to the Communication subsystem, trajectory error to AOCS, delta-v remaining to Propulsion, and finally processor temperature and memory integrity to the OBC subsystem.

After generation of the nominal telemetry specific to each mission, we injected faults by defining a unique set of windows for each mission during which faults would occur for a given subsystem. These windows ranged from 3 to 13% of the total arbitrary simulated mission length of 5000 time points, and their sizes were chosen based on the likely relative frequency of faults occurring in each subsystem informed by mission-specific operational assumptions regarding which system would be more critical or susceptible to degradation. For example, during Earth Observation missions, the Communication subsystem represents a critical component due to the requirement of regular data transmission and ground contact. Therefore, a larger fault window was assigned for this subsystem than for Propulsion, which is typically used less frequently once orbital operations have been established. On the other hand, propulsion during lunar missions is heavily relied-upon due to repeated trajectory adjustments and descent maneuvers, and as such is likely to generate more faults than the other subsystems, meaning the fault window for this subsystem was made appropriately large.

The faults themselves were injected by modelling the various telemetry variables as showing either gradual degradation or sudden discrete deviations in their values from nominal

patterns, according to the scenario that was most sensible in the context of each variable. For example, solar array degradation was modelled as a gradual continuous decline, whereas the antenna in the deep space scenario was designed to exhibit discrete failure. Both major and minor faults were injected to produce variability of the severity of anomalies that the health monitoring system would encounter. This approach ensured that the injected anomalies resulted in more realistic telemetry patterns suitable for evaluation by both the rule-based and machine learning anomaly detection methods.

Health Monitoring

Having generated the mission telemetry including faults, the first stage of the health monitoring system applies an Isolation Forest algorithm to identify anomalous system behaviour. This algorithm was chosen as it is an unsupervised machine learning method, making it suitable for spacecraft applications where representative labelled fault datasets are often unavailable, and for its ability to detect anomalies by the principle of isolating outliers, datapoints that deviate from the statistical characteristics of nominal data. This was implemented in addition to a rule-based anomaly detection method to allow the system to isolate abnormal behaviour within the spacecraft even if no nominal operational telemetry ranges are violated. Thus, this layer of the Asclepius system architecture is able to detect more subtle degradations and can act as an early warning signal that a fault may be present.

An independent Isolation Forest model was trained on nominal data from each spacecraft subsystem. This allowed them to learn the normal multi-dimensional behaviour of each subsystem, without evaluating any faults as normal. A contamination rate of 3% was set during model training to define the approximate proportion of anomalous observations and allow the models to establish appropriate decision boundaries between nominal and anomalous telemetry. After training, the models were applied to the mission telemetry for their respective subsystem. For every telemetry sample the model was exposed to, it produced an anomaly score, which quantified the degree to which the sample differed from nominal patterns, as well as a binary prediction classifying each sample as either anomalous or normal. These data were subsequently joined with the mission telemetry dataset to be fed into the next stage of the health monitoring pipeline.

The complementary rule-based fault detection process sought to apply deterministic engineering knowledge by comparing telemetry values with defined operational thresholds. These rules were designed to represent realistic failure conditions rather than simple statistical outliers; for example, battery voltage below the defined threshold indicates voltage degradation, and excessive reaction wheel speed indicates loss of attitude control. For each variable contained within the generated mission telemetry datasets, we defined bounds in relation to the nominal value range which, if violated, would result in the system flagging an anomaly. Multiple rules may be triggered simultaneously, both between and within subsystems, allowing for complex subsystem failures to be represented by combinations of individual fault conditions. For example, a critical thermal subsystem failure may generate anomalously high readings of the battery, bus, and payload temperatures at the same time.

Rather than producing only a binary rule-based fault indicator, Asclepius calculates a rule-based severity score for each subsystem, representing the number of telemetry variables whose operational limits have been exceeded at a given time. This provides a more informative

assessment of subsystem degradation by distinguishing isolated threshold violations from more widespread subsystem failures. A binary rule-based issue flag is then also derived from this score as a simple indicator of whether at least one engineering rule has been violated in a subsystem. Alongside the rule-based assessment, the Isolation Forest models independently produced machine learning anomaly flags indicating whether subsystem behaviour deviated significantly from nominal operating patterns. The outputs of the two detection methods were subsequently fused to produce a combined subsystem issue flag, which was activated whenever either the rule-based detector or the machine learning detector identified abnormal behaviour. These combined issue flags served as internal indicators of subsystem health and formed the primary inputs to the subsequent event extraction stage.

In addition to the combined issue flags, Asclepius generated subsystem health status classifications intended for operator interpretation. Unlike the combined issue flags, these status classifications were derived solely from deterministic rule-based detections and therefore represented confirmed engineering threshold violations rather than statistically anomalous behaviour. This distinction allows operators to differentiate between explicit subsystem faults and machine learning anomalies that may warrant investigation before being interpreted as genuine failures. Together, the combined issue flags and subsystem status classifications provide both an internal representation of subsystem health for downstream processing and an intuitive summary of spacecraft condition for the operator.

Finally, an overall spacecraft health status was computed by aggregating the health of all subsystems. This assessment considered both confirmed rule-based subsystem faults and the presence of machine learning anomalies, while assigning greater importance to mission-critical subsystems such as Power, Communication, Propulsion, and Mobility. The resulting spacecraft health classification provides a concise summary of the vehicle's operational condition at each telemetry time step and serves as a high-level indicator for both event extraction and operator situational awareness.

Event Extraction

While the health monitoring stage identifies abnormal subsystem behaviour at every telemetry time step, this information is not immediately useful to a spacecraft operator, who would instead benefit more from a concise list of operational events. The event extraction stage therefore converts the continuous stream of subsystem health indicators into discrete engineering events representing sustained spacecraft faults. In order to achieve this, the Asclepius system continuously monitors the combined subsystem issue flags generated during health monitoring. To reduce false positives caused by transient sensor noise or single statistical outliers, an anomaly is only considered to have begun after remaining active for a minimum number of 10 consecutive telemetry samples. Likewise, an event is not considered complete until the subsystem has returned to nominal behaviour for a specified recovery period. This hysteresis prevents rapid oscillation between normal and anomalous states and produces more stable event boundaries.

Once an event has been identified, the algorithm determines its associated fault type by examining which rule-based engineering conditions were active during the event. Since multiple operational limits may be exceeded simultaneously, a single event may contain several fault types, allowing complex subsystem failures to be represented as combinations of individual

engineering faults. If no deterministic engineering rules are active but the machine learning detector continues to identify statistically abnormal behaviour, the event is labelled as an ‘unknown anomaly’, indicating that abnormal subsystem behaviour has been detected without matching any predefined failure mode.

Each extracted event is subsequently assigned both an operational severity and a confidence level. Operational severity is determined from the engineering significance of the detected fault type together with the duration of the event, allowing prolonged subsystem degradation to be prioritised over short-lived transient abnormalities. Specifically, if the anomalous event is detected by both the rule-based and ML-based detectors, and also lasts more than 200 time steps, the event is labelled as ‘critical’, regardless of the subsystem in which it occurred. For the critical subsystems, however, this limit is set to the lower value of 100 to reflect the greater threat such a fault can pose to the spacecraft’s operations. If a dual-detected anomaly satisfies neither of these conditions, its severity is labelled simply as ‘warning’. In the case that the event is detected only by the rule-based detector, it is given ‘warning’ severity, and if only identified by the ML detector, a severity of ‘investigation’ is assigned. The event’s confidence reflects the level of agreement between the two detection methods: events identified by both the rule-based and machine learning detectors are classified as ‘High’ confidence, while those detected by only one method are assigned ‘Medium’ confidence. Events unsupported by either detector are considered ‘Low’ confidence, although these occur rarely in practice. Thus, for every event, the log produces information on its associated subsystem, event type (e.g. reaction wheel saturation), whether or not it was isolated by the rule-based detector, and by the ML-based detector, confidence, start time, end time, duration, and its severity.

Finally, neighbouring events occurring within the same subsystem are merged when separated by only a defined short interval of 30 time steps of nominal behaviour. In such cases, the combined event was assigned with the higher of the individual events’ severity classifications. This merging prevents persistent failures that momentarily recover from being fragmented into multiple independent events, producing a cleaner and more operationally meaningful event log. The resulting structured event list forms the primary input to the subsequent Large Language Model (LLM) reasoning engine, the final output of the system, which interprets the events in the context of the relevant mission scenario and generates engineering recommendations for the spacecraft operator.

Mission Event Logs

The following mission event logs were generated by Asclepius for the three mission scenarios. For each anomaly detected, they include information on its subsystem, event type, whether it was detected by the deterministic rules and/or the Isolation Forest, the confidence in the anomaly’s presence, its start and end times, duration, and its severity.

Earth observation:

Subsystem	Event type	Rule	ML	Confidence	Start	End	Duration	Severity
AOCS	['Attitude accuracy degradation']	True	True	High	2770	2906	136	Critical
AOCS	['Unknown anomaly']	False	True	Medium	3284	3335	51	Investigate

AOCS	['Reaction wheel saturation', 'Gyroscope drift anomaly']	True	True	High	3362	3760	398	Critical
Coms	['Signal degradation']	True	True	High	318	361	43	Warning
Coms	['Signal degradation', 'Data rate reduction', 'Packet loss spike']	True	True	High	389	909	520	Critical
Coms	['Signal degradation']	True	True	High	1614	1629	15	Warning
Coms	['Ground station visibility lost']	True	True	High	3849	3854	5	Warning
Coms	['Ground station visibility lost']	True	True	High	4256	4275	19	Warning
OBC	['CPU overload']	True	True	High	3984	4060	76	Warning
OBC	['CPU overload', 'Memory saturation']	True	True	High	4319	4659	340	Critical
Optics	['Camera overheating']	True	True	High	1286	1609	323	Critical
Optics	['Unknown anomaly']	False	True	Medium	2275	2320	45	Investigate
Optics	['Image quality degradation']	True	True	High	2328	2406	78	Warning
Optics	['Imaging power overload']	True	True	High	3298	3408	110	Warning
Optics	['Unknown anomaly']	False	True	Medium	4052	4107	55	Investigate
Power	['Voltage degradation', 'Current overload', 'Solar array degradation']	True	True	High	829	1260	431	Critical

Power	['Unknown anomaly']	False	True	Medium	1446	1459	13	Investigate
Power	['Voltage degradation']	True	False	Medium	2883	2888	5	Warning
Propulsion	['Thruster overheating', 'Extreme thrust output']	True	True	High	3518	3659	141	Critical
Thermal	['Unknown anomaly']	False	True	Medium	1259	1332	73	Investigate
Thermal	['Bus overheating']	True	True	High	1387	1456	69	Warning
Thermal	['Battery overheating', 'Bus overheating', 'Payload overheating']	True	True	High	2776	3010	234	Critical

Lunar exploration:

Subsystem	Event type	Rule	ML	Confidence	Start	End	Duration	Severity
AOCS	['Gyroscope drift anomaly']	True	True	High	1677	1709	32	Warning
AOCS	['Reaction wheel saturation', 'Gyroscope drift anomaly']	True	True	High	2819	3259	440	Critical
Coms	['Earth visibility lost']	True	True	Medium	321	349	28	Warning
Coms	['Signal degradation', 'Data rate reduction', 'Packet loss spike', 'Earth visibility lost']	True	True	High	2049	2259	210	Critical
Coms	['Earth visibility lost', 'Low link margin']	True	True	High	3535	3558	23	Warning
Mobility	['Extreme terrain slope']	True	True	High	434	452	18	Warning

Mobility	['Unknown anomaly']	False	True	Medium	1525	1609	84	Investigation
Mobility	['Wheels slipping', 'Extreme terrain slope', 'Mobility degradation']	True	True	High	4119	4202	83	Warning
OBC	['CPU overload', 'Memory saturation']	True	True	High	3991	4055	64	Warning
OBC	['CPU overload', 'Memory saturation']	True	True	High	4546	4759	213	Critical
Power	['Voltage degradation', 'Solar array degradation']	True	True	High	917	952	35	Warning
Power	['Voltage degradation', 'Current overload', 'Solar array degradation']	True	True	High	1532	2009	477	Critical
Power	['Solar array degradation']	True	True	High	3610	3684	74	Warning
Propulsion	['Extreme thrust output']	True	True	High	3257	3359	102	Critical
Propulsion	['Thruster overheating', 'Extreme thrust output']	True	True	High	3798	4409	611	Critical
Thermal	['Battery overheating', 'Bus overheating', 'Payload overheating']	True	True	High	772	1009	237	Critical
Thermal	['Unknown anomaly']	True	True	Medium	1241	1456	215	Warning
Thermal	['Excessive heater power']	True	True	High	2810	2920	110	Warning
Thermal	['Unknown anomaly']	False	True	Medium	3049	3104	55	Investigate

Deep space:

Subsystem	Event type	Rule	ML	Confidence	Start	End	Duration	Severity
AOCS	['Reaction wheel saturation', 'Gyroscope drift anomaly']	True	True	High	1052	1509	457	Critical
AOCS	['Gyroscope drift anomaly']	True	True	High	1677	1709	32	Warning
Coms	['Signal degradation']	True	False	Medium	319	325	6	Warning
Coms	['Signal degradation', 'Antenna failure']	True	True	High	1757	2153	396	Critical
Coms	['Unknown anomaly']	False	True	Medium	3276	3299	23	Investigate
Coms	['Unknown anomaly']	True	True	Medium	3555	4010	455	Critical
OBC	['CPU overload', 'Memory saturation']	True	True	High	2555	2909	354	Critical
OBC	['CPU overload', 'Memory saturation']	True	True	High	3991	4052	61	Warning
Power	['Voltage degradation', 'Solar array degradation']	True	True	High	906	959	53	Warning
Power	['Unknown anomaly']	False	True	Medium	1631	1667	36	Investigate
Power	['Unknown anomaly']	False	True	Medium	1709	1818	109	Investigate
Power	['Unknown anomaly']	False	True	Medium	1873	1879	6	Investigate
Power	['Voltage degradation', 'Current overload', 'Solar array degradation']	True	True	High	3300	3759	459	Critical
Power	['Solar array degradation']	True	True	High	4916	4924	8	Warning
Propulsion	['Thruster overheating',	True	True	High	546	759	213	Critical

	'Extreme thrust output']							
Propulsion	['Extreme thrust output']	True	True	High	4084	4202	118	Critical
Thermal	['Unknown anomaly']	True	True	Medium	1245	1455	210	Warning
Thermal	['Battery overheating', 'Bus overheating', 'Payload overheating']	True	True	High	2034	2708	674	Critical

Mission Reasoning

The final stage of the Asclepius system pipeline provides mission-level reasoning and decision support through the use of a locally hosted LLM. The decision-support component employs the open-source Llama 3.1 large language model, hosted locally using the Ollama inference framework, enabling mission reasoning to be performed entirely offline without reliance on external cloud services. Rather than interpreting raw telemetry directly, the LLM instance operates on the structured event log produced during the event extraction stage, allowing its analysis to be grounded in higher-level engineering events rather than individual sensor measurements. Each extracted event is first converted into a structured textual representation containing the affected subsystem, detected fault type, whether the anomaly was identified by the rule-based and/or machine learning detectors, the assigned confidence and severity classifications, and the event timing and duration. This event log is then embedded within a carefully engineered prompt together with mission-specific contextual information describing the spacecraft, mission objectives, subsystem priorities, and the set of operator actions available for that mission scenario.

The prompt instructs the LLM to act as an autonomous spacecraft mission operations assistant and generate a structured engineering report. To maximise factual reliability, explicit constraints are incorporated into the prompt to ensure that every conclusion is supported by either the event log, the mission profile, or logical operational consequences of the observed failures. The model is specifically instructed not to invent spacecraft hardware, mission phases, environmental conditions, or fault causes that are unsupported by the available evidence, and to explicitly state *'Further investigation required'* whenever insufficient information exists to draw a conclusion. Additional guidance is provided regarding the interpretation of event confidence, severity, subsystem interactions, and event prioritisation to promote consistent reasoning across different mission scenarios.

The generated report follows a fixed engineering structure consisting of an executive summary, overall mission health assessment, prioritised fault ranking, separation of confirmed failures from uncertain anomalies, subsystem interaction analysis, likely causes, recommended operator actions, and an overall mission outlook. It begins with an executive summary, providing a concise overview of the current mission scenario, overall spacecraft health, mission risk, and

the number of confirmed failures and uncertain anomalies. This is followed by a mission health assessment, which evaluates the operational condition of the spacecraft and identifies the subsystems posing the greatest risk to mission success. The report then presents a prioritised fault assessment, ranking detected events according to their operational importance by considering event severity, confidence, duration, and mission-specific priorities. Events are subsequently categorised into confirmed failures and uncertain anomalies, allowing operators to distinguish between deterministic engineering faults and statistical abnormalities that require further investigation. A subsystem interaction assessment then examines whether failures in one subsystem may plausibly contribute to degradation in another, highlighting potential cascading effects where supported by the event data. The report also provides a discussion of likely causes, carefully distinguishing evidence-based conclusions from hypotheses requiring additional investigation. Finally, the LLM generates a prioritised list of recommended operator actions, together with the reasoning and confidence associated with each recommendation, before concluding with an overall mission outlook summarising immediate operational risks, possible cascading failures, and the subsystem requiring the highest level of monitoring.

Thus, by combining deterministic event extraction with contextual natural language reasoning, this final stage transforms technical fault information into concise operational guidance suitable for spacecraft operators. The result is an interpretable decision-support system capable of summarising complex spacecraft health information, prioritising operator attention, and providing mission-specific recommendations while remaining grounded in the detected telemetry evidence, and minimising unfounded speculation. (For the mission-specific engineering reports generated by the LLM, see separate file)

Conclusion

This project demonstrates the feasibility of combining multiple artificial intelligence techniques with conventional engineering methods to create an integrated spacecraft health monitoring and decision support system. Rather than relying solely on deterministic threshold-based monitoring or purely data-driven anomaly detection, Asclepius employs a hybrid architecture in which synthetic telemetry generation, rule-based fault detection, machine learning anomaly detection, event extraction, and large language model reasoning operate together as a unified health management pipeline. The resulting system is capable of transforming low-level telemetry measurements into high-level operational guidance suitable for spacecraft mission operators.

The synthetic telemetry generation framework proved valuable in enabling realistic testing of the system despite the limited availability of publicly accessible spacecraft telemetry datasets. By developing separate telemetry models for Earth Observation, Lunar Exploration, and Deep Space missions, the project was able to evaluate the health monitoring framework across spacecraft with substantially different operational profiles and subsystem priorities. Injecting both gradual degradation faults and abrupt subsystem failures produced realistic mission scenarios that exercised the complete monitoring pipeline.

The hybrid health assessment approach represents one of the principal strengths of the system. Traditional spacecraft fault management systems rely heavily on predefined engineering thresholds, which provide high-confidence identification of known failure modes but are inherently unable to recognise previously unseen behaviour. Conversely, machine learning methods such as Isolation Forests are capable of identifying statistically abnormal subsystem

behaviour but cannot determine whether such behaviour necessarily represents an engineering fault. By combining deterministic rule-based monitoring with machine learning anomaly detection, Asclepius benefits from the complementary strengths of both approaches. Explicit engineering failures are detected with high confidence while statistically unusual subsystem behaviour can still be highlighted for further investigation, improving overall spacecraft situational awareness.

The event extraction stage further enhances the operational usefulness of the monitoring system by converting continuous anomaly detections into discrete engineering events. Requiring anomalies to persist for a minimum duration before generating an event reduces sensitivity to transient fluctuations and sensor noise, while merging neighbouring events prevents prolonged subsystem degradation from being fragmented into multiple independent incidents. The assignment of event confidence and severity provides additional operational context, enabling mission operators to distinguish between confirmed subsystem failures and lower-confidence anomalies requiring continued monitoring.

The final reasoning stage illustrates the potential application of modern large language models within spacecraft operations. Instead of interpreting raw telemetry directly, the LLM operates upon structured engineering events generated by the preceding stages, allowing it to produce mission-level assessments, prioritise faults, identify subsystem interactions, and recommend operator actions. Extensive prompt engineering was employed to constrain the model to evidence contained within the mission profile and event log, reducing unsupported reasoning and encouraging objective engineering assessments. While large language models cannot replace traditional flight software or human operators, they demonstrate considerable potential as decision-support tools capable of rapidly synthesising complex spacecraft health information into concise operational reports.

Taken together, the project demonstrates a practical application of artificial intelligence within the context of spacecraft operations. By integrating machine learning anomaly detection with deterministic engineering rules and natural language processing, Asclepius provides a complete end-to-end software framework for spacecraft health monitoring and mission decision support. Although the system has been validated using synthetic telemetry rather than operational spacecraft data, the modular architecture would readily support future extension to real mission datasets, additional anomaly detection techniques, more sophisticated reasoning models, or onboard autonomous health management. The project therefore illustrates how contemporary AI technologies can complement already-established spacecraft engineering practices to improve autonomy, operator situational awareness, and ultimately mission resilience.